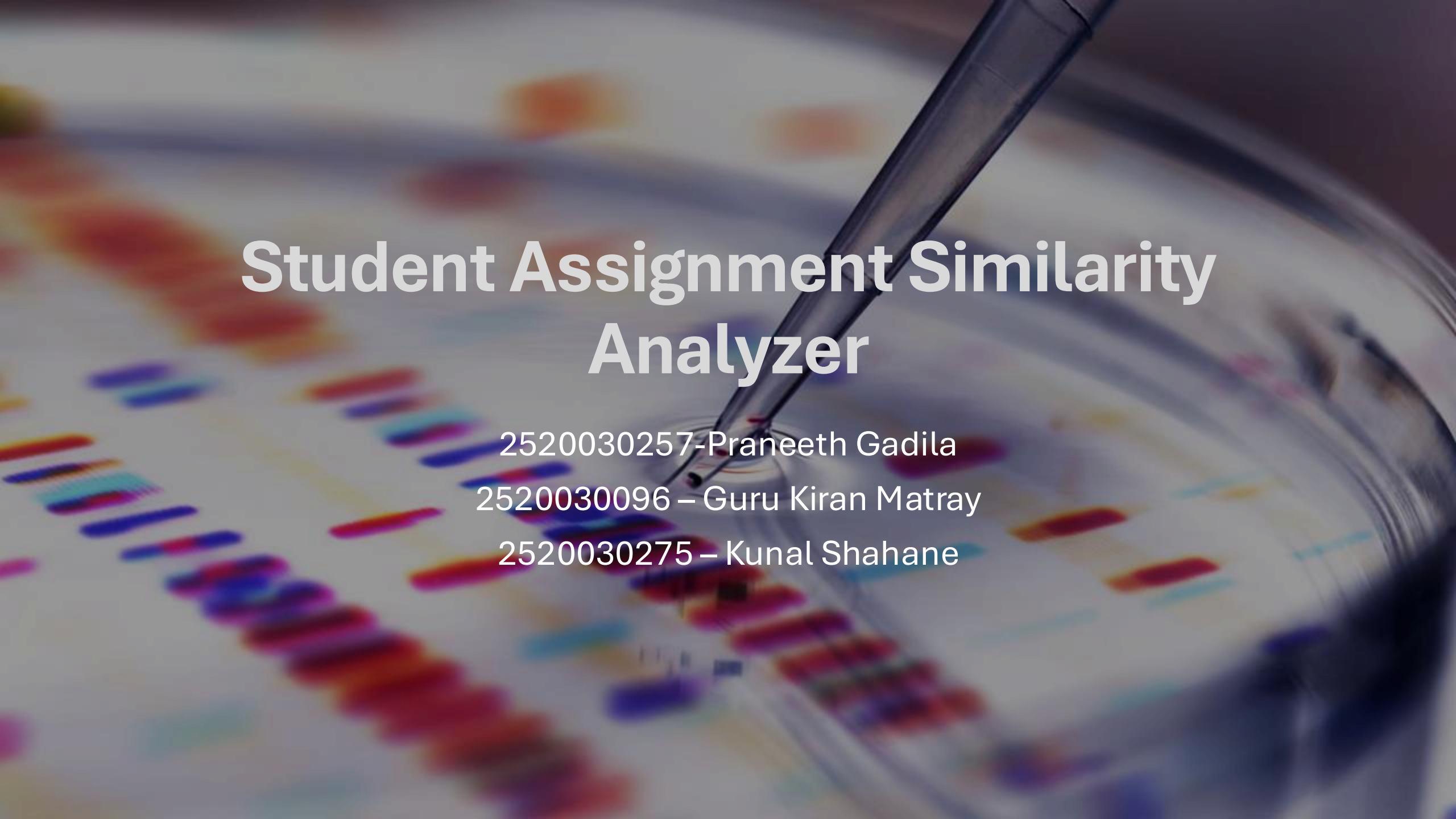


The background of the slide is a blurred image of a DNA gel electrophoresis experiment. A glass pipette is shown in the upper right, dispensing a liquid into one of the wells of a gel. The gel itself is filled with various colored bands (red, blue, green, yellow) representing different DNA fragments. The text is overlaid on this background.

Student Assignment Similarity Analyzer

2520030257-Praneeth Gadila

2520030096 – Guru Kiran Matray

2520030275 – Kunal Shahane

Abstract & Problem Statement

ABSTRACT

- A Java-based system for detecting plagiarism and similarity in **student TXT/PDF assignments**.
- Automatically compares each new submission with **all previous assignments**.
- Uses **String Algorithms and Dynamic Programming** to detect exact and modified copied content.
- Generates similarity percentages and identifies the most similar submissions.

PROBLEM STATEMENT

- Manual comparison of multiple assignments is **time-consuming and difficult to scale**.
- Students may modify copied content through **word changes, insertions, deletions, or transpositions**.
- An automated system is needed to efficiently compare submissions and flag suspicious similarities.
- The system classifies assignments using a **10% similarity threshold: $\leq 10\%$ Original, $> 10\%$ Copied**.

Description

- Java-based **console application** for detecting assignment similarity and plagiarism.
- Accepts **TXT and PDF** assignment files.
- Extracts and preprocesses text from submitted documents.
- Uses **TF-IDF and Cosine Similarity** to calculate document similarity.
- Compares new assignments with previously submitted reference documents.
- Generates a **similarity percentage** for each comparison.
- Classifies assignments as **Original or Copied** using a predefined threshold.
- Demonstrates practical application of **Data Structures & Algorithms**.
- **Technologies:** Java 17+, Apache PDFBox, TF-IDF, Cosine Similarity.

Tools, Technologies & Expected Outcome

Tools & Technologies

- **Language:** Java
- **GUI:** Java Swing
- **IDE:** Visual Studio Code
- **File Formats:** TXT & PDF
- **PDF Processing:** Apache PDFBox
- **String Algorithms:** Naïve, KMP, Z-Function, Rabin-Karp, Aho-Corasick
- **Suffix Algorithms:** Suffix Array, LCP & Kasai
- **Dynamic Programming:** Levenshtein, Wagner-Fischer, Damerau-Levenshtein
- **Similarity:** TF-IDF & Cosine Similarity
- **Data Structures:** Trie, HashMap, Arrays, Lists

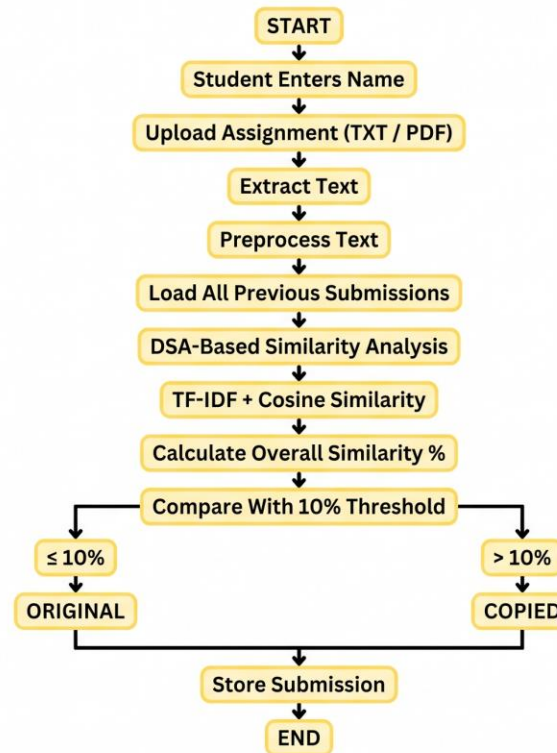
Tools, Technologies & Expected Outcome

Expected Outcome

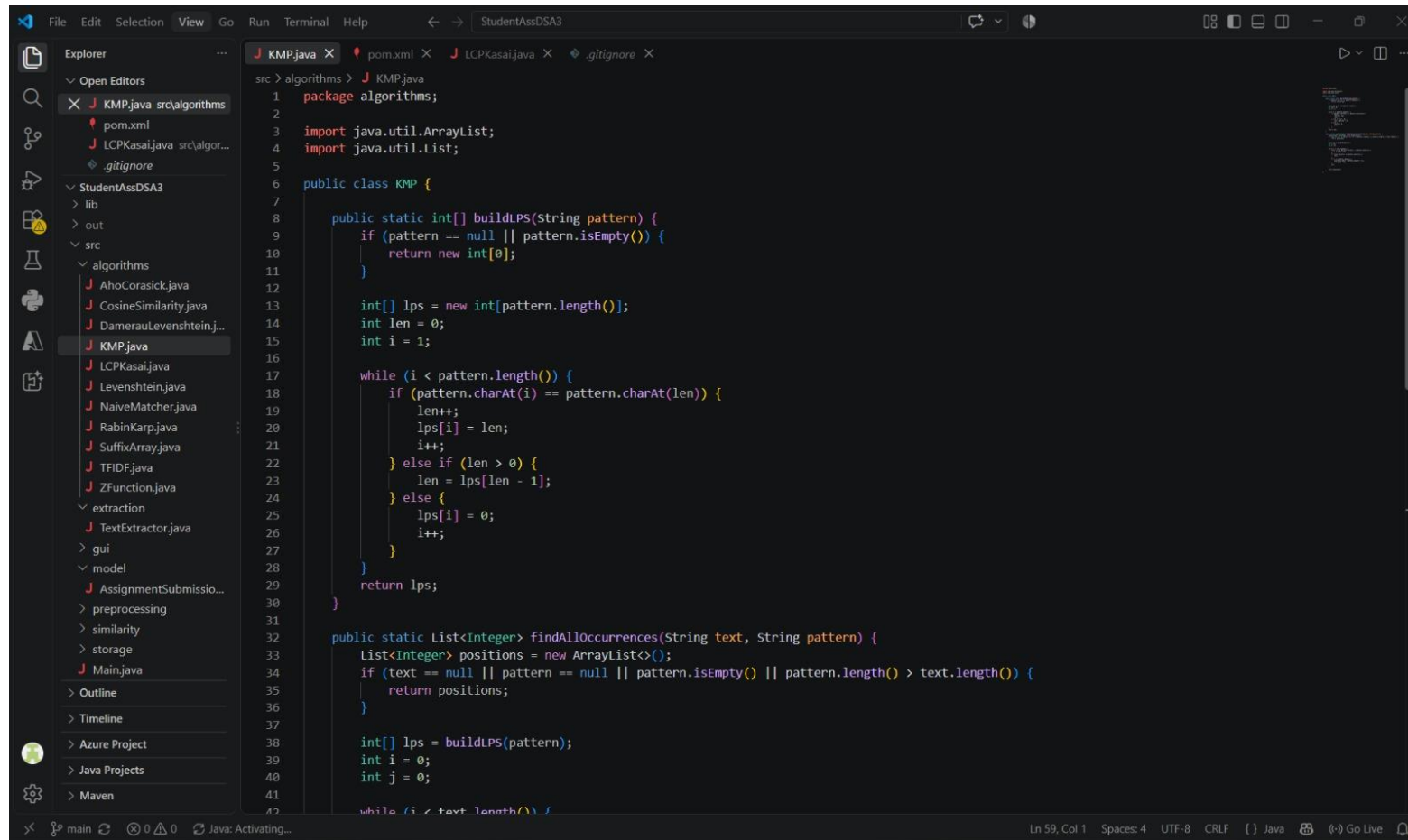
- Automatic comparison with **all previous assignments**.
- Accurate identification of **exact and modified copied content**.
- Pairwise similarity percentage for every previous submission.
- Identification of the **highest-similarity assignment**.
- Automatic **Original/Copied classification** using the 10% threshold.
- Reduced manual plagiarism checking and improved scalability for large numbers of submissions.

Workflow

STUDENT ASSIGNMENT SIMILARITY ANALYZER



Code implementation

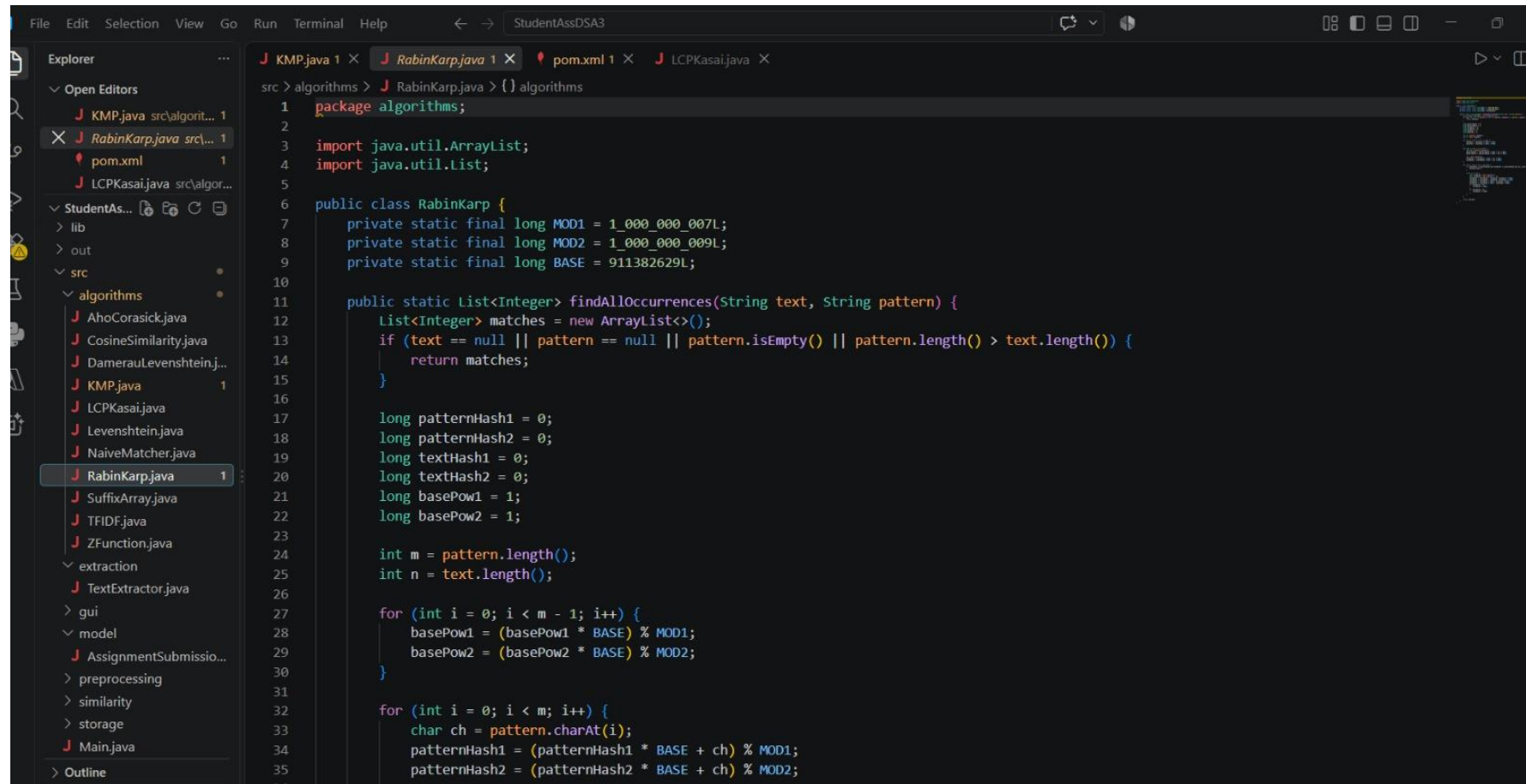


The screenshot shows an IDE window titled "StudentAssDSA3" with the following components:

- Explorer:** A sidebar on the left showing the project structure. The "src" directory is expanded, showing subdirectories like "algorithms" and "extraction". The file "KMP.java" is selected under "algorithms".
- Editor:** The main area displays the code for "KMP.java". The code implements the KMP algorithm, including the "buildLPS" method and the "findAllOccurrences" method.
- Terminal:** A terminal window at the bottom shows the command "Java: Activating..." and the status "Ln 59, Col 1 Spaces: 4 UTF-8 CRLF {} Java".

```
1 package algorithms;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class KMP {
7
8     public static int[] buildLPS(String pattern) {
9         if (pattern == null || pattern.isEmpty()) {
10             return new int[0];
11         }
12
13         int[] lps = new int[pattern.length()];
14         int len = 0;
15         int i = 1;
16
17         while (i < pattern.length()) {
18             if (pattern.charAt(i) == pattern.charAt(len)) {
19                 len++;
20                 lps[i] = len;
21                 i++;
22             } else if (len > 0) {
23                 len = lps[len - 1];
24             } else {
25                 lps[i] = 0;
26                 i++;
27             }
28         }
29         return lps;
30     }
31
32     public static List<Integer> findAllOccurrences(String text, String pattern) {
33         List<Integer> positions = new ArrayList<>();
34         if (text == null || pattern == null || pattern.isEmpty() || pattern.length() > text.length()) {
35             return positions;
36         }
37
38         int[] lps = buildLPS(pattern);
39         int i = 0;
40         int j = 0;
41         while (i < text.length()) {
42             while (i < text.length() && j < lps.length() && text.charAt(i) != pattern.charAt(j)) {
43                 j = lps[j];
44             }
45             if (j == lps.length()) {
46                 positions.add(i);
47                 i++;
48             } else {
49                 i++;
50                 j++;
51             }
52         }
53         return positions;
54     }
55 }
```

Code implementation



The screenshot shows an IDE window with the following components:

- Explorer:** A sidebar on the left showing the project structure. The 'src' directory is expanded, and the 'algorithms' subdirectory is selected. The file 'RabinKarp.java' is highlighted.
- Editor:** The main window displays the code for 'RabinKarp.java'. The code is as follows:

```
1 package algorithms;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class RabinKarp {
7     private static final long MOD1 = 1_000_000_007L;
8     private static final long MOD2 = 1_000_000_009L;
9     private static final long BASE = 911382629L;
10
11     public static List<Integer> findAllOccurrences(String text, String pattern) {
12         List<Integer> matches = new ArrayList<>();
13         if (text == null || pattern == null || pattern.isEmpty() || pattern.length() > text.length()) {
14             return matches;
15         }
16
17         long patternHash1 = 0;
18         long patternHash2 = 0;
19         long textHash1 = 0;
20         long textHash2 = 0;
21         long basePow1 = 1;
22         long basePow2 = 1;
23
24         int m = pattern.length();
25         int n = text.length();
26
27         for (int i = 0; i < m - 1; i++) {
28             basePow1 = (basePow1 * BASE) % MOD1;
29             basePow2 = (basePow2 * BASE) % MOD2;
30         }
31
32         for (int i = 0; i < m; i++) {
33             char ch = pattern.charAt(i);
34             patternHash1 = (patternHash1 * BASE + ch) % MOD1;
35             patternHash2 = (patternHash2 * BASE + ch) % MOD2;
```

Output

The screenshot shows the 'STUDENT ASSIGNMENT SIMILARITY ANALYZER' application window. The interface is divided into several sections:

- Current Submission:** Contains input fields for 'Student Name' (kunal), 'Assignment File' (IsJAVAARRAYS.txt), and 'File Type' (TXT). A 'Choose File' button is next to the file input.
- Reference / Comparison Documents (Current Session):** Includes buttons for '+ Add Reference Files', 'Remove Selected', and 'Clear References'. Below, a list shows two reference documents: '1. foreach.txt [TXT]' and '2. passwords.txt [TXT]'.
- Analysis Result:** Displays the results of the similarity analysis:
 - Current Submission: JAVAARRAYS.txt
 - Reference Documents: 2
 - Files Compared: 2
 - OVERALL SIMILARITY: 13.96%**
 - THRESHOLD: 10.00%**
 - FINAL STATUS: COPIED** (highlighted in a red box)
- Summary:** A text area at the bottom right contains a summary of the analysis:

```
Student      : kunal
Current File  : JAVAARRAYS.txt
Previous Docs : 2
Files Compared : 2

OVERALL SIMILARITY : 13.96%
THRESHOLD          : 10.00%
FINAL STATUS       : COPIED
```

At the bottom of the application window, there are buttons for 'CHECK SIMILARITY', 'CLEAR ALL', 'History', and 'Algorithms'.

Geotag images

